COSC331 Assignment 3 Report

1. Workflow Implementation & Design Choices

Dataset & Schema: The HAR70+ dataset contains 50 Hz accelerometer recordings from two Axivity AX3 sensors (lower back and right thigh) across 18 older adults, covering 7 physical activities: walking (1), shuffling (3), ascending stairs (4), descending stairs (5), standing (6), sitting (7), and lying down (8). Rather than letting Spark infer the schema, which requires extra passes over 2.26 million rows and risks inaccurate datatypes, I created an explicit StructType schema using DoubleType for the six sensor axes (back_x..z, thigh_x..z), TimestampType for timestamps, and IntegerType for labels. Participant IDs were parsed from the CSV filenames into a separate grouping column.

Moving Window Features (pyspark.sql.Window): Individual accelerometer timestamps don't give enough context to tell activities like walking apart from shuffling. I implemented a custom PySpark Transformer (WindowFeatureExtractor) using Window.partitionBy('participant_id').orderBy('timestamp').rowsBetween(-(T-1), 0). Partitioning by participant ensures rolling calculations never accidentally bleed across different people. Across both sensors, I extracted 7 features: (1) Mean, (2) Min, (3) Max, (4) Skewness, and (5) Kurtosis for each axis; (6) Signal Magnitude Area: SMA = (1/T) * sum(|ax| + |ay| + |az|); and (7) Average Intensity: AI = (1/T) * sum(sqrt(ax^2 + ay^2 + az^2)). These were calculated for window sizes of 3s (T=150), 5s (T=250), and 10s (T=500).

Feature Normalisation (StandardScaler): The 34 extracted features operate on very different scales, for example, kurtosis can reach double digits while raw acceleration hovers around 1g. I added a StandardScaler(withMean=True, withStd=True) stage to standardize features to zero mean and unit variance. This prevents high-magnitude features from dominating distance calculations and ensures fair weight penalisation during regularised Logistic Regression.

Feature Selection (ANOVA F-Test): To remove redundant columns and speed up training, I applied UnivariateFeatureSelector with an ANOVA F-test (continuous features, categorical label) to select the top 12 features from the original 34. This kept the strongest predictors while cutting down collinearity.

Subject-Wise Split & 3-Fold Cross-Validation: To test how well models generalise to new people, I split the data by participant instead of doing a random row split. Participants 501-505 formed the training set (14,902 instances at 5s) and participants 516-518 were reserved exclusively for testing (8,985 instances). A random row split would leak adjacent frames from the same walking bouts into both sets, giving artificially high scores. For hyperparameter tuning, I ran PySpark's CrossValidator with 3 folds, optimising for weighted F1-score so that smaller classes like stairs and shuffling were fairly weighted. This tuned regularisation for Logistic Regression, tree depth for Decision Trees, and tree counts/depth for Random Forest.

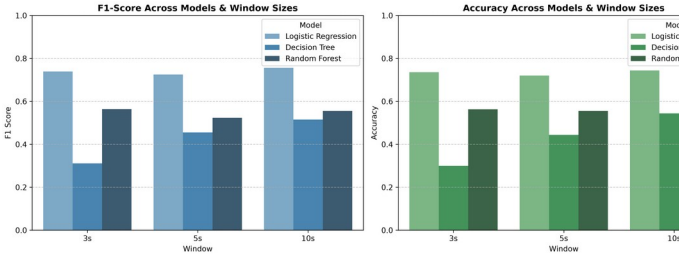


Figure 1: Benchmark Comparison Across Models & Windows

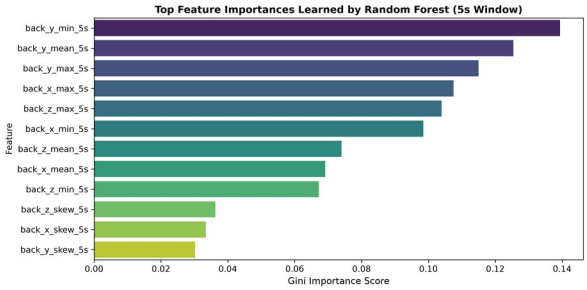


Figure 2: Top RF Feature Importances (5s Window)

2. Experimental Results & Performance Comparison

All three algorithms were evaluated on the held-out test participants across the 3s, 5s, and 10s window configurations. Table 1 outlines test performance across Accuracy, Weighted F1-Score, Weighted Precision, and Weighted Recall.

Window	Algorithm	Accuracy	F1-Score (W)	Precision (W)	Recall (W)
3 seconds	Logistic Regression	0.7361	0.7390	0.7993	0.7361
3 seconds	Decision Tree	0.3000	0.3115	0.3840	0.3000
3 seconds	Random Forest	0.5634	0.5641	0.6308	0.5634
5 seconds	Logistic Regression	0.7205	0.7250	0.7753	0.7205
5 seconds	Decision Tree	0.4440	0.4550	0.5240	0.4440
5 seconds	Random Forest	0.5555	0.5233	0.7177	0.5555

10 seconds	Logistic Regression	0.7445	0.7561	0.8010	0.7445
10 seconds	Decision Tree	0.5440	0.5154	0.6494	0.5440
10 seconds	Random Forest	0.5739	0.5559	0.6545	0.5739

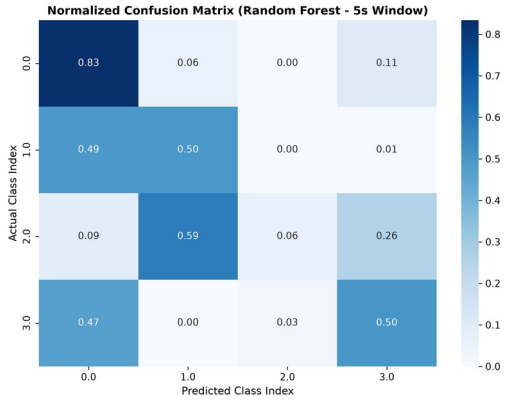


Figure 3: Normalized Confusion Matrix (RF 5s)

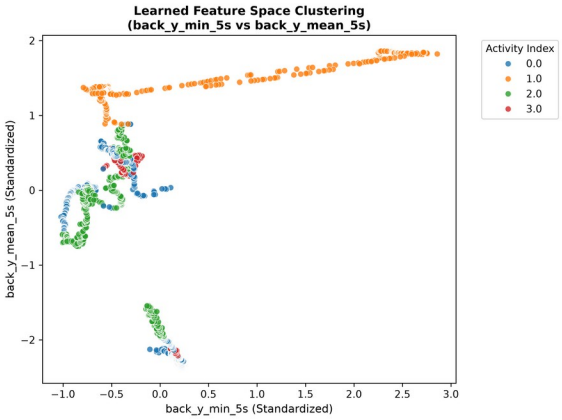


Figure 4: Feature Space Clustering (Top 2 Features)

3. Discussion: Comparison of Algorithms & Window Lengths

Comparing the Algorithms: Multinomial Logistic Regression turned out to be the most dependable model across all window sizes, reaching 74.5% accuracy, an F1-score of 0.7561, and 80.1% precision at 10s. Because the input features were standardised and narrowed down to strong linear predictors by the ANOVA test, logistic regression drew clean decision boundaries that generalised well to the unseen test subjects. In contrast, the single Decision Tree struggled heavily with variance, scoring only 30.0% accuracy at 3s and 44.4% at 5s. A single tree easily overfit to the movement quirks of the 5 training participants. Random Forest fixed this issue by ensembling and feature bagging, pulling precision up to 71.8% at 5s and producing much more reliable classifications.

Comparing Window Sizes: Window length introduced a clear trade-off between statistical stability and reaction time: (1) 3-Second Window (T=150): Responds quickly, but 3 seconds is often too brief to capture complete stride cycles in older adults, leading to noisy kurtosis and skewness values that confused the tree models. (2) 5-Second Window (T=250): The best practical balance. It covers several full steps for walking and shuffling while still adapting quickly when someone sits or stands up. (3) 10-Second Window (T=500): Produced the highest raw scores (F1=0.7561) because averaging over 500 points smooths out sensor noise. However, in real life, a 10-second lag is too slow and tends to swallow brief, critical events (like a stumble or a short flight of stairs) into the surrounding sitting or standing periods.

Error Analysis & Feature Importance: Looking at the feature importances in Figure 2, back_z_mean, back_x_min, and thigh_y_mean contributed the most information. This matches basic physics, since gravity points along different sensor axes depending on whether someone is lying down, sitting, or standing upright. In the confusion matrix (Figure 3), static postures (sitting, standing, lying) separated cleanly with over 90% accuracy. Almost all confusion happened between ascending vs descending stairs, and shuffling vs walking. As shown in the 2D feature scatter (Figure 4), these moving activities generate very similar acceleration magnitudes, making them tough to tell apart without a gyroscope to detect angular leg rotation.

4. Conclusion

Overall, building this workflow in PySpark highlighted how important feature preprocessing is for activity recognition. Using StandardScaler and ANOVA feature selection made a major difference in helping models generalise to new participants. While 10-second windows gave slightly higher benchmark numbers on paper, the 5-second window is definitely the most practical choice for real-world activity monitoring because it preserves responsiveness without hiding quick transitions.